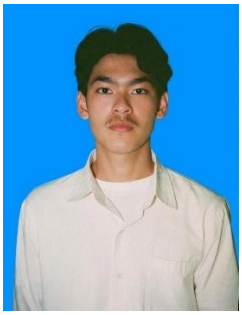
LAPORAN PRAKTEK

Pemrograman Struktur Data

MUHAMMAD RAMDHAN KHOTAMI PUTRA (225443045)



TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI

Politeknik Manufaktur Bandung

Tahun Ajaran 2025/2026

DAFTAR ISI

LAPORAN PRAKTIKUM

LAPORAN PRAKTEK i

DAFTAR ISI ii

LAPORAN PRAKTIKUM QUEUE 1

Queue dengan Linear Array 1

Analisis Kode: 1

Flowchart Queue dengan Linear Array 6

Queue dengan Circular Array 7

Analisis Kode: 8

Flowchart Queue dengan Circular Array 12

Queue dengan Double Linked List 13

Analisis Code 13

Flowchart Queue dengan Double Linked List 17

LAPORAN PRAKTIKUM QUEUE

Queue dengan Linear Array

```cpp
#include <iostream>
#define MAX 20
using namespace std;

struct Queue
{
    int front, rear, data[MAX];
} Q;

bool isFull()
{
    return Q.rear == MAX;
}

bool isEmpty()
{
    return Q.rear == 0;
}

void printQueue()
{
    if (isEmpty())
    {
        cout << "Antrian kosong" << endl;
    }
    else
    {
        cout << "QUEUE : ";
        for (int i = Q.front; i < Q.rear; i++)
        {
            cout << Q.data[i] << ((Q.rear - 1 == i) ? "" : ",");
            cout << endl;
        }
    }
}

void enqueue()
{
    if (isFull())
    {
        cout << "Antrian penuh" << endl;
    }
    else
    {
        int data;
        cout << "Masukkan Data : ";
        cin >> data;
        Q.data[Q.rear] = data;
        Q.rear++;
        cout << "Data ditambahkan\n";
        printQueue();
    }
}

void dequeue()
{
    if (isEmpty())
    {
        cout << "Antrian masih kosong" << endl;
    }
    else
    {
        cout << "Mengambil data \" " << Q.data[Q.front] << " \"..." << endl;
        for (int i = Q.front; i < Q.rear; i++)
            Q.data[i] = Q.data[i + 1];
        Q.rear--;
        printQueue();
    }
}

int main()
{
    int choose;
    do
    {
        cout << "---\n"
             << " Menu Pilihan\n"
             << "---\n"
             << "[1] Enqueue \n"
             << "[2] Dequeue\n"
             << "[3] Keluar \n\n"
             << "---\n"
             << "Masukkan pilihan : ";
        cin >> choose;
        switch (choose)
        {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        default:
            cout << "Pilihan tidak tersedia";
            break;
        }
    } while (choose != 3);
    return 0;
}
```

Analisis Kode:

```cpp
#define MAX 20
```

Maksimalnya data pada antrian adalah 20

```
struct Queue
{
    int front, rear, data[MAX];
} Q;
```

front = penunjuk data yang paling depan

rear = penunjuk posisi terakhir data

data[MAX] = array untuk disimpan dalam isi antrian

Q = variable global antrian

```
bool isFull()
{
    return Q.rear == MAX;
}
```

Kode ini menunjukan bahwa jika rear == 20 maka akan menghasilkan output “Antrian penuh”

```
bool isEmpty()
{
    return Q.rear == 0;
}
```

Kode ini akan berfungsi jika rear == 0 dan akan menghasilkan output “Antrian masih kosong”

```
void printQueue()
{
    if (isEmpty())
    {
        cout << "Antrian kosong" << endl;
    }
    else
    {
        cout << "QUEUE : ";
        for (int i = Q.front; i < Q.rear; i++) Penjelasan: (loop dari depan front=0 sampai
sebelum rear)
        {
            cout << Q.data[i] << ((Q.rear - 1 == i) ? "" : ","); Penjelasan: (jika bukan
data terakhir maka akan dikasih koma di ujungnya, sebaliknya jika data terakhir maka tidak
menngunakan koma)
            cout << endl;
        }
    }
}
```

Penjelasan kode ada dibagian kodenya.

```
void enqueue()
{
    if (isFull())
    {
        cout << "Antrian penuh" << endl;
```

```
    }
    else
    {
        int data;
        cout << "Masukkan Data : ";
        cin >> data;
        Q.data[Q.rear] = data; Penjelasan: (data dimasukan ke index)
        Q.rear++; Penjelasan: ( rear naik 1 )
        cout << "Data ditambahkan\n";
        printQueue();
    }
}
```

Penjelasan kode ada dibagian kodenya.

```
void dequeue()
{
    if (isEmpty())
    {
        cout << "Antrian masih kosong" << endl;
    }
    else
    {
        cout << "Mengambil data \" " << Q.data[Q.front] << " \"..." << endl;
        for (int i = Q.front; i < Q.rear; i++)
            Q.data[i] = Q.data[i + 1]; Penjelasan ( kode loop ini berfungsi untuk menggeser
data kesebelah kiri )
        Q.rear--; Penjelasan (kode ini berfungsi untuk menurunkan rear karena data telah
berkurang)
        printQueue();
    }
}
```

Penjelasan kode ada dibagian kodenya.

```
int main()
{
    int choose;
    do
    {
        cout << "---\n"
             << " Menu Pilihan\n"
             << "---\n"
             << "[1] Enqueue \n"
             << "[2] Dequeue\n"
             << "[3] Keluar \n\n"
             << "---\n"
             << "Masukkan pilihan : ";
        cin >> choose;
        switch (choose)
        {
        case 1:
            enqueue();
            break;
        case 2:
            dequeue();
            break;
        default:
            cout << "Pilihan tidak tersedia";
            break;
        }
    } while (choose != 3);
    return 0;
}
```

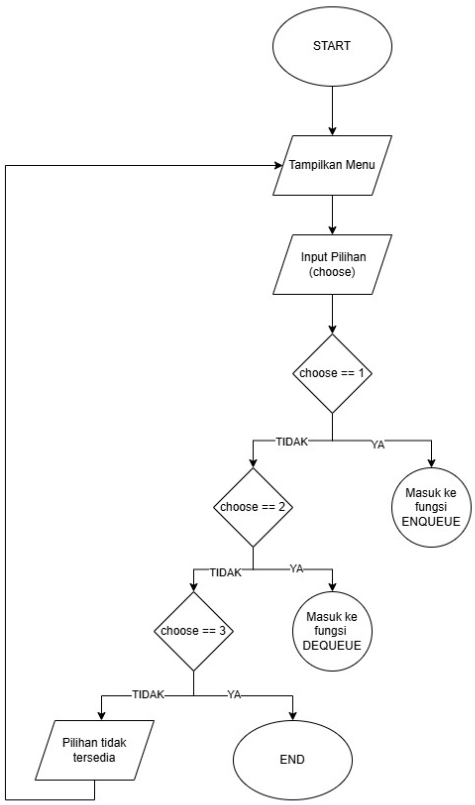
fungsi main ini berfungsi sebagai mengatur aksi sesuai pilihan user

Output dari program:

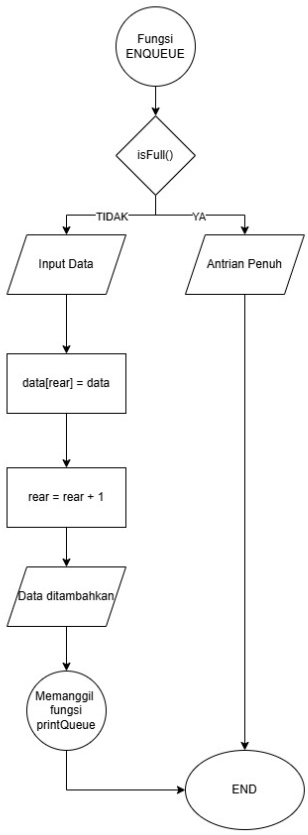
```
[1] Enqueue
[2] Dequeue
[3] Keluar

---
Masukkan pilihan : 1
Masukkan Data : 21
Data ditambahkan
QUEUE : 21
---
 Menu Pilihan
---
[1] Enqueue
[2] Dequeue
[3] Keluar

---
Masukkan pilihan : 1
Masukkan Data : 11
Data ditambahkan
QUEUE : 21,
11
---
 Menu Pilihan
---
[1] Enqueue
[2] Dequeue
[3] Keluar

---
Masukkan pilihan : 1
Masukkan Data : 44
Data ditambahkan
QUEUE : 21,
11,
44
---
 Menu Pilihan
---
[1] Enqueue
[2] Dequeue
[3] Keluar
```

Flowchart Queue dengan Linear Array

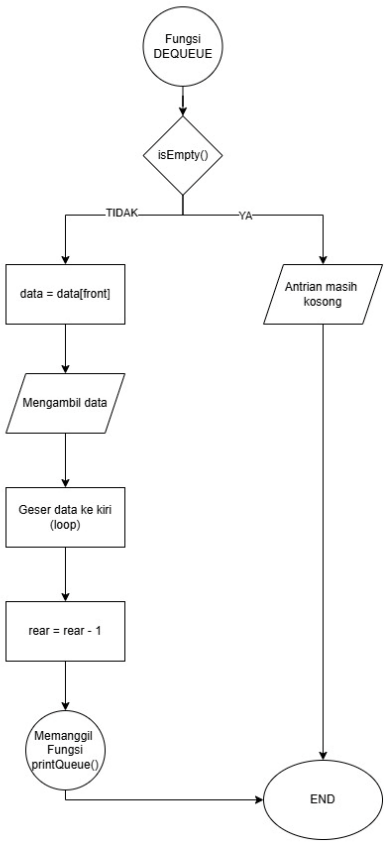
FUNGSI MAIN



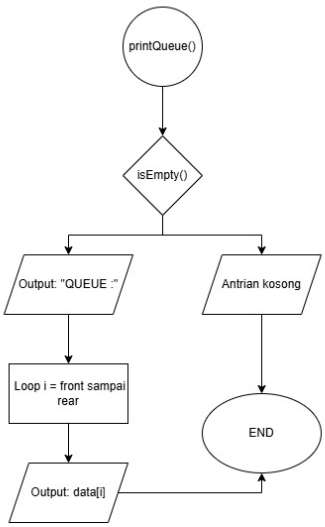
FUNGSI ENQUEUE



FUNGSI DEQUEUE



FUNGSI PRINT QUEUE



Queue dengan Circular Array

```
#include <iostream>
#define MAX 5
using namespace std;

class Circular_Queue
{
private:
    int *queue_arr;
    int front, rear;

public:
    Circular_Queue()
    {
        queue_arr = new int[MAX];
        rear = front = -1;
    }

    void insert(int item)
    {
        if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
        {
            cout << "Queue Penuh \n";
            return;
        }
        if (front == -1)
        {
            front = 0;
            rear = 0;
        }
        else
        {
            if (rear == MAX - 1)
                rear = 0;
            else
                rear = rear + 1;
        }
        queue_arr[rear] = item;
        display();
    }

    void del()
    {
        if (front == -1)
        {
            cout << "Queue elements : Kosong..!\n";
            return;
        }
        cout << "Element deleted from queue is : " << queue_arr[front] << endl;
        if (front == rear)
        {
            front = -1;
            rear = -1;
        }
        else
        {
            if (front == MAX - 1)
                front = 0;
            else
                front = front + 1;
        }
        display();
    }

    void display()
    {
        int front_pos = front, rear_pos = rear;
        if (front == -1)
        {
            cout << "Queue elements : Kosong..!\n";
            return;
        }
        cout << "Queue elements :\n";
        if (front_pos <= rear_pos)
        {
            while (front_pos <= rear_pos)
            {
                cout << queue_arr[front_pos] << " ";
                front_pos++;
            }
        }
        else
        {
            while (front_pos <= MAX - 1)
            {
                cout << queue_arr[front_pos] << " ";
                front_pos++;
            }
            front_pos = 0;
            while (front_pos <= rear_pos)
            {
                cout << queue_arr[front_pos] << " ";
                front_pos++;
            }
        }
        cout << endl;
    }
};

int main()
{
    int choice, item;
    Circular_Queue cq;
    do
    {
        cout << "1.Insert\n";
        cout << "2.Delete\n";
        cout << "3.Quit\n";
        cout << "Input pilihan : ";
        cin >> choice;
        switch (choice)
        {
        case 1:
            cout << "Ketik nilai yang akan di queue : ";
            cin >> item;
            cq.insert(item);
            break;
        case 2:
            cq.del();
            break;
        case 3:
            break;
        default:
            cout << "Pilihan salah.!\n";
        }
    } while (choice != 3);

    return 0;
}
```

Analisis Kode:

```
#define MAX 5
```

Maksimal kapasistas data adalah 5

```
class Circular_Queue
{
private:
    int *queue_arr;
    int front, rear;
```

queue_arr = pointer keluar

front = posisi data depan

rear = posisi data belakang

```
    Circular_Queue()
    {
        queue_arr = new int[MAX];
        rear = front = -1;
    }
```

Antrian kosong akan ditandai dengan front = -1, rear = -1

Penjelasan kode pada fungsi insert():

```
if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
```

1. front = 0 dan rear = MAX-1 (penuh normal)
2. front = rear + 1 (rear "ngejar" front karena muter)

```
 if (front == -1)
        {
            front = 0;
            rear = 0;
        }
```

Data pertama masuk ke index 0 maka front & rear jadi 0

```
if (rear == MAX - 1)
                rear = 0;
```

Kalau rear udah mentok maka akan balik ke index 0

```
else
                rear = rear + 1;
```

Kalau data belum mentok data akan maju seperti biasa

```
queue_arr[rear] = item;
```

Simpan data di posisi rear

Penjelasan Kode pada fungsi del():

```
if (front == -1)
```

Antrian kosong

```
if (front == rear)
        {
            front = -1;
            rear = -1;
        }
```

Antrian menjadi kosong Kembali

```
            if (front == MAX - 1)
                front = 0;
            else
                front = front + 1;
```

Menggeser posisi front tanpa menggeser data

```
void display()
    {
        int front_pos = front, rear_pos = rear;
        if (front == -1)
        {
            cout << "Queue elements : Kosong..!\n";
            return;
        }
        cout << "Queue elements :\n";
        if (front_pos <= rear_pos)
        {
            while (front_pos <= rear_pos)
            {
                cout << queue_arr[front_pos] << " ";
                front_pos++;
            }
        }
        else
        {
            while (front_pos <= MAX - 1)
            {
                cout << queue_arr[front_pos] << " ";
                front_pos++;
            }
            front_pos = 0;
            while (front_pos <= rear_pos)
            {
                cout << queue_arr[front_pos] << " ";
                front_pos++;
            }
```

```
        }
        cout << endl;
    }
```

Fungsi display() digunakan untuk menampilkan isi dari circular queue sesuai urutan logis antrian. Pertama, fungsi mengecek apakah queue kosong dengan kondisi front == -1; jika benar, maka akan menampilkan pesan bahwa antrian kosong. Jika tidak kosong, fungsi akan mencetak elemen-elemen queue dengan dua kemungkinan kondisi: jika front <= rear, berarti data tersusun normal (tidak melingkar), sehingga elemen ditampilkan dari indeks front sampai rear secara berurutan. Namun jika front > rear, berarti queue dalam kondisi melingkar, sehingga proses pencetakan dilakukan dalam dua tahap, yaitu dari front hingga indeks terakhir array (MAX - 1), lalu dilanjutkan dari indeks 0 hingga rear. Dengan cara ini, meskipun posisi data di array tidak berurutan secara fisik, output tetap mencerminkan urutan antrian yang benar (FIFO).

```
int main()
{
    int choice, item;
    Circular_Queue cq;
    do
    {
        cout << "1.Insert\n";
        cout << "2.Delete\n";
        cout << "3.Quit\n";
        cout << "Input pilihan : ";
        cin >> choice;
        switch (choice)
        {
        case 1:
            cout << "Ketik nilai yang akan di queue : ";
            cin >> item;
            cq.insert(item);
            break;
        case 2:
            cq.del();
            break;
        case 3:
            break;
        default:
            cout << "Pilihan salah.!\n";
        }
    } while (choice != 3);

    return 0;
}
```

Fungsi main() berperan sebagai pengendali utama jalannya program dengan menampilkan menu interaktif kepada pengguna dalam sebuah perulangan do-while. Program akan terus berjalan selama pengguna belum memilih opsi

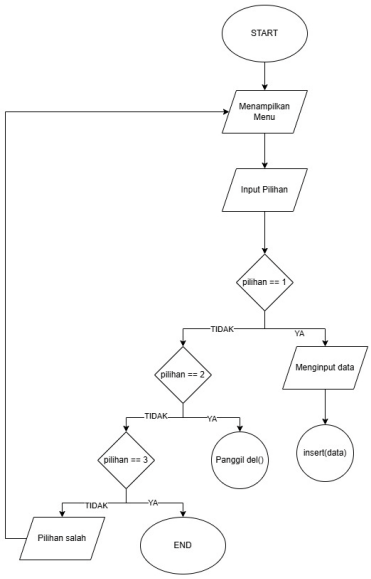
keluar (3). Di dalam loop, pengguna diminta memasukkan pilihan, yaitu 1 untuk menambahkan data ke dalam antrian (enqueue), 2 untuk menghapus data dari antrian (dequeue), dan 3 untuk keluar dari program. Pilihan tersebut kemudian diproses menggunakan struktur switch, di mana jika pengguna memilih 1, program akan meminta input nilai lalu memanggil fungsi insert(), sedangkan jika memilih 2, program akan langsung memanggil fungsi del(). Jika input tidak sesuai dengan pilihan yang tersedia, maka akan ditampilkan pesan kesalahan. Dengan alur ini, fungsi main() mengatur interaksi pengguna sekaligus menghubungkan semua operasi pada circular queue hingga program dihentikan.

Output Program:

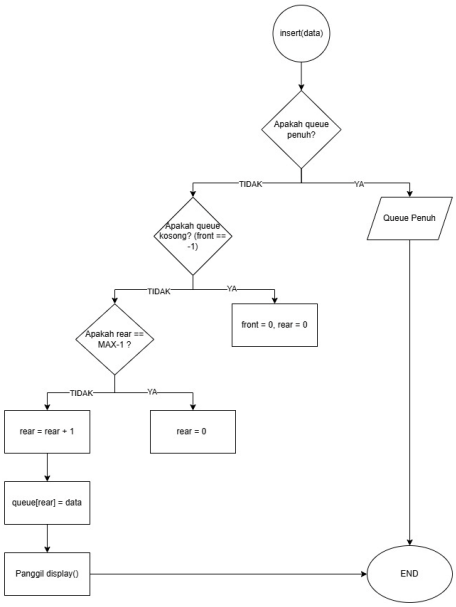
```
1.Insert
2.Delete
3.Quit
Input pilihan : 1
Ketik nilai yang akan di queue : 2
Queue elements :
2
1.Insert
2.Delete
3.Quit
Input pilihan : 1
Ketik nilai yang akan di queue : 6
Queue elements :
2 6
1.Insert
2.Delete
3.Quit
Input pilihan : 1
Ketik nilai yang akan di queue : 7
Queue elements :
2 6 7
1.Insert
2.Delete
3.Quit
Input pilihan : 2
Element deleted from queue is : 2
Queue elements :
6 7
1.Insert
2.Delete
3.Quit
Input pilihan :
```

Flowchart Queue dengan Circular Array

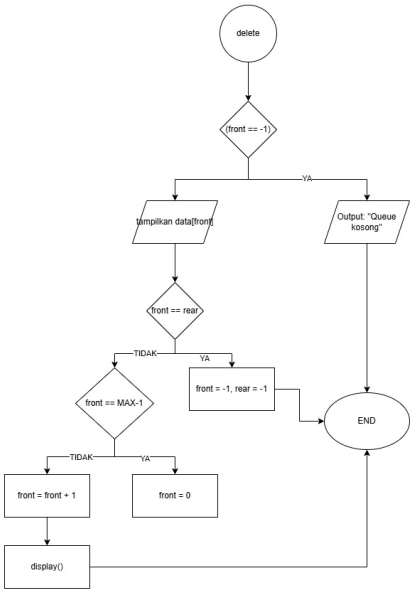
FLOWCHART MAIN



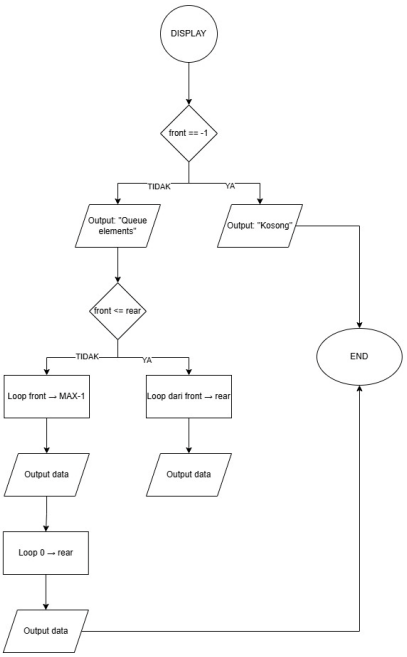
FLOWCHART INSERT



FLOWCHART DELETE



FLOWCHART DISPLAY



Queue dengan Double Linked List

Analisis Code

```
#include <iostream>
using namespace std;
struct Node
{
    int data;
    Node *next;
};
```

Kode ini digunakan untuk mendefinisikan satu elemen dalam queue berbasis linked list. Variabel `data` berfungsi untuk menyimpan nilai, sedangkan `next` adalah pointer yang menunjuk ke node berikutnya. Dengan adanya pointer ini, setiap node dapat saling terhubung membentuk sebuah rantai data.

```
class Queue
{
    struct Node *head, *tail;
```

Pada bagian ini, `head` digunakan untuk menunjuk elemen paling depan dalam queue, sedangkan `tail` menunjuk elemen paling belakang. Kedua pointer ini sangat penting karena menentukan dari mana data diambil dan ke mana data ditambahkan.

```
public:
    Queue()
    {
        head = tail = NULL;
    }
```

Kode ini berfungsi untuk menginisialisasi queue dalam keadaan kosong. Nilai `NULL` menandakan bahwa belum ada node yang tersimpan dalam queue.

```
    void enQueue();
    void deQueue();
    void displayQueue();
    void menu();
    int elem;
    int choice;
};
void Queue::enQueue()
{
    cout << "Enter your element to be inserted to the queue: ";
    cin >> elem;
    Node *pointer = new Node;
    pointer->data = elem;
    pointer->next = NULL;
```

Pada bagian ini, program menerima input dari pengguna lalu membuat node baru. Data yang dimasukkan disimpan ke dalam node tersebut, dan pointer `next` diatur ke `NULL` karena node ini akan ditempatkan di posisi paling belakang.

```
    if (head == NULL)
    {
        head = pointer;
    }

    else
        tail->next = pointer;
    tail = pointer;
```

Logika ini mengatur posisi node baru dalam queue. Jika queue masih kosong, maka node baru langsung menjadi elemen pertama sehingga `head` menunjuk ke node tersebut. Namun jika sudah ada isi, maka node baru akan disambungkan ke belakang melalui `tail->next`. Setelah itu, `tail` diperbarui agar menunjuk ke node terbaru. Dengan cara ini, data selalu masuk dari belakang antrian.

```
    cout << "Element has been inserted in the queue!" << endl;
}
void Queue::deQueue()
{
    if (head == NULL)
```

Kode ini digunakan untuk mengecek apakah queue kosong. Jika kosong, maka tidak ada data yang bisa dihapus.

```
    {
        cout << "Queue is empty!" << endl;
    }
    else
    {
        Node *temp = head;
        head = head->next;
        delete temp;
```

Jika queue tidak kosong, maka node paling depan akan dihapus. Caranya dengan menyimpan node yang ditunjuk oleh `head` ke variabel sementara, kemudian `head` dipindahkan ke node berikutnya. Setelah itu node lama dihapus dari memori. Proses ini memastikan bahwa data yang pertama masuk akan menjadi yang pertama keluar.

```
    }
}
void Queue::displayQueue()
{
    Node *pointer1 = head;
    if (head == NULL)
    {
        cout << "Queue is empty!" << endl;
    }
    else
    {
        cout << "Elements of your QUEUE!" << endl;
        while (pointer1 != NULL)
        {
            cout << pointer1->data << endl;
            pointer1 = pointer1->next;
        }
```

Kode ini digunakan untuk menelusuri seluruh isi queue mulai dari `head` hingga akhir. Pointer bantu akan bergerak dari satu node ke node berikutnya, dan setiap data yang dilewati akan ditampilkan. Perulangan berhenti ketika pointer mencapai `NULL`, yang menandakan akhir dari linked list.

```
        cout << "End" << endl;
    }
}
void Queue::menu()
{
    while (1)
    {
        cout << "================================" << "\n";
        cout << " 1. Queue" << "\n";
        cout << " 2. Dequeue" << "\n";
        cout << " 3. Display Queue" << '\n';
        cout << " 4. Exit" << '\n';
        cout << "==========================" << '\n';
        cout << "\nEnter your choice: ";
        cin >> choice;
        switch (choice)
```

Kode ini membuat program berjalan terus menerus sampai pengguna memilih keluar. Input dari pengguna akan menentukan fungsi mana yang dijalankan.

```
        {
        case 1:
            enQueue();
            break;
        case 2:
            deQueue();
            break;
        case 3:
            displayQueue();
            break;
        case 4:
            return;
```

Setiap pilihan akan memanggil fungsi sesuai kebutuhan, yaitu menambah data, menghapus data, menampilkan data, atau keluar dari program.

```
        default:
            cout << "Enter choice(1-4)";
            break;
        }
    }
}
int main()
{
    Queue frank;
    frank.menu();
}
```

Kode ini membuat sebuah objek dari class `Queue` dan menjalankan fungsi `menu()` sebagai titik awal program. Dari sinilah seluruh alur program berjalan.

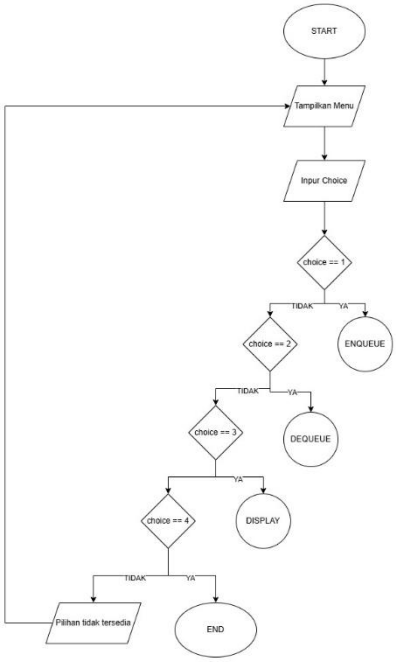
Secara keseluruhan, kode ini menunjukkan bagaimana queue dapat diimplementasikan menggunakan linked list, di mana penambahan dan penghapusan data dilakukan tanpa perlu menggeser elemen seperti pada array, sehingga lebih efisien dan fleksibel.

Output Program:

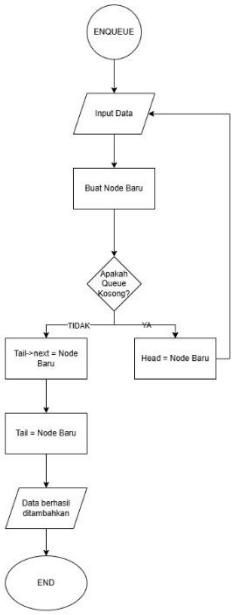
```
PS C:\Users\user\OneDrive\Documents\POLMAN\TRIN\Pemrograman Struktur data\Minggu ke 2> ./main
 2. Dequeue
 3. Display Queue
 4. Exit
=========================

Enter your choice: 1
Enter your element to be inserted to the queue: 7
Element has been inserted in the queue!
==============================
 1. Queue
 2. Dequeue
 3. Display Queue
 4. Exit
=========================

Enter your choice: 1
Enter your element to be inserted to the queue: 6
Element has been inserted in the queue!
==============================
 1. Queue
 2. Dequeue
 3. Display Queue
 4. Exit
=========================

Enter your choice: 3
Elements of your QUEUE!
7
6
End
==============================
 1. Queue
 2. Dequeue
 3. Display Queue
 4. Exit
=========================

Enter your choice: 2
==============================
 1. Queue
 2. Dequeue
 3. Display Queue
 4. Exit
=========================

Enter your choice: 3
```

Flowchart Queue dengan Double Linked List

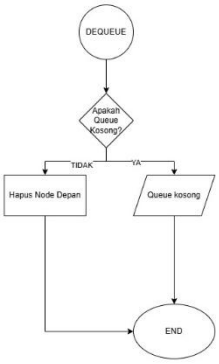
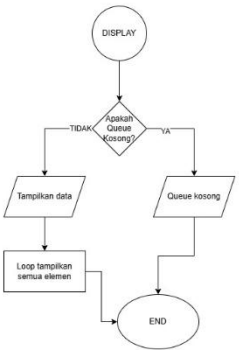
FLOWCHART MENU



FLOWCHART ENQUEUE



FLOWCHART ENQUEUE



FLOWCHART ENQUEUE